Endpoint Log Anomaly Detection Bot Using Isolation Forest

Introduction

This project implements an endpoint log anomaly detection system using raw Windows Security Event Logs. The code parses and preprocesses log data, extracts behavioral features, and applies machine learning techniques to identify abnormal activities.

An Isolation Forest model is used to detect anomalies, while K-Means clustering assigns risk levels to events. The system helps identify suspicious users or processes and supports real-time security monitoring.

Methodology

The methodology of this project is based on a structured pipeline for detecting anomalies in endpoint logs using machine learning and graph analysis. The process begins with collecting raw Windows Security Event Logs and transforming them from JSON data format into a structured format suitable for analysis. The code follows a step-by-step approach that includes data parsing, preprocessing, feature engineering, clustering, anomaly detection, and visualization.

First, raw logs are parsed to extract relevant security attributes such as Time Created, event IDs, process names, and IP addresses. After cleaning and preprocessing the data, meaningful behavioral features are engineered to represent user and system activity patterns. These features are then used in both unsupervised clustering (K-Means) and anomaly detection (Isolation Forest). The results are further analyzed to classify high-risk users or IP addresses and to construct a graph-based representation of relationships between entities.

Finally, the methodology supports real-time monitoring through a bot that continuously analyzes new logs and generates alerts for suspicious activity.

Log Parsing and Preprocessing Approach

The parsing process focuses on converting raw Windows Security Event Logs, provided in JSON format, into a structured pandas DataFrame. Each log entry is treated as a single event and parsed to extract key components such as TimeCreated, eventID, ProcessName, Channel, and other related fields. The timestamp or TimeCreated fields are converted into datetime objects to allow time-based analysis.

During preprocessing, missing or inconsistent values are handled to ensure data quality for example, duplicate rows were removed to avoid data bias, NAN values were populated with specific values based on the EventID. Categorical features such as Channel, ProcessName, and host are encoded into numerical representations using LabelEncoder to be suitable for machine learning models. Numerical features are normalized using standard scaling techniques to prevent features with large ranges from dominating the analysis such as EventID and TimeCreated values. This preprocessing step ensures that the dataset is clean, consistent, and optimized for clustering and anomaly detection models.

Clustering for Risk Level Assignment

In this project, clustering is applied to group endpoint log events based on similarities in user behavior and system activity in order to assign meaningful risk levels. After parsing and preprocessing the Windows Security Event Logs, numerical features extracted from the dataset are first standardized and then reduced using **Principal Component Analysis (PCA)**. PCA is applied to reduce dimensionality while preserving the most important variance in the data, which improves clustering performance and visualization. The transformed features are then used as input to the K-Means clustering algorithm.

To determine the optimal number of clusters, the Elbow Method is applied by evaluating the within-cluster sum of squares for different values of *k*. Based on the elbow point observed in the plot, the number of clusters is set to three, representing distinct behavioral groups within the endpoint logs. The K-Means algorithm then groups log events by minimizing intra-cluster distances and maximizing inter-cluster separation. Each cluster represents a different activity profile, ranging from normal system behavior to more irregular and intensive activity patterns.

After clustering, the three clusters are mapped to predefined risk levels: low, medium, and high risk. Clusters containing normal and repetitive system events with stable behavior are classified as low risk, while clusters showing moderate deviations are labeled as medium risk. The cluster associated with abnormal behavior patterns—such as excessive UserEventFrequancy, unusual process execution, or suspicious login activity—is classified as high risk. This clustering-based risk assignment adds contextual meaning to the anomaly detection process and helps prioritize security alerts and support effective incident response.

Top IP/User Classification

The Top IP/User classification step focuses on identifying the most active and potentially suspicious entities within the endpoint log dataset. Using the processed Windows Security Event Logs, the code aggregates events by EventID and IP addresses to calculate UserEventFrequancy. This aggregation allows the system to rank users or IPs based on the number of EventID they generate, highlighting entities that exhibit unusually high activity compared to the rest of the dataset. The top

10 events are then selected for further analysis, as high event volume is often correlated with abnormal or malicious behavior.

Once the top entities are identified, their activity is analyzed in combination with clustering and anomaly detection results. For each of the top 10 events, the code evaluates associated features such as failed login attempts, abnormal process executions, and anomaly scores produced by the Isolation Forest model. Based on these indicators, each entity is classified into potential threat categories, such as unauthorized access for frequent failed authentication events or malware-related activity for unusual process behavior. This classification is rule-based and derived directly from observed log patterns rather than predefined labels.

By combining event frequency with anomaly detection and clustering-based risk levels, this approach provides a prioritized and interpretable view of the most suspicious users or IP addresses in the system. Security analysts can quickly focus on high-risk entities that repeatedly appear in anomalous events, improving investigation efficiency and supporting faster incident response.

Graph Analysis

Graph analysis is used in this project to model and analyze relationships between IP addresses based on their occurrence order in the endpoint logs. Using the Networkx library, a graph is constructed where each node represents an IP address extracted from the processed dataset. An edge is added between two IP addresses when they appear consecutively in the log sequence, capturing a temporal relationship between events. This approach allows the graph to reflect how IP addresses are connected through event flow rather than treating them as isolated entities. The resulting graph is visualized to provide an intuitive overview of IP interactions within the system.

After building the graph, degree centrality is calculated for each IP address to measure its importance within the network. Degree centrality represents how frequently an IP is connected to other IPs, indicating how active or influential it is in the event sequence. The IPs are then ranked based on their degree centrality values, and the top 10 most central IPs are identified as potentially suspicious entities. The graph is further visualized using a spring layout, with edge labels displaying the degree centrality values of connected nodes. This analysis helps highlight IP addresses that play a central role in the log activity and may represent infected hosts, scanners, or sources of abnormal behavior.

Bot Functionality (Real-Time Detection)

The real-time security bot is designed to continuously monitor endpoint logs and analyze new events as they are generated. After training, all required machine learning artifacts including the StandardScaler, PCA model, Isolation Forest, K-Means model, selected important features, and the risk mapping are saved using joblib and loaded by the bot during runtime. The bot relies on the watchdog library to observe changes in a specified log file, and it automatically triggers analysis whenever a new log entry is detected. Each new log line is parsed from JSON format into a structured DataFrame, ensuring compatibility with the preprocessing and feature engineering steps used during training.

For every incoming event, the bot updates a running counter to track user activity frequency and prepares the feature vector by aligning it with the most important features used by the models. The data is standardized, reduced using PCA, and then passed to both the K-Means and Isolation Forest models. K-Means assigns the event to a risk cluster that is mapped to a risk level, while the Isolation Forest predicts whether the event is anomalous and computes an anomaly score. If an event belongs to a high-risk cluster or is flagged as an anomaly, the bot generates an immediate alert containing detailed information such as the user, event count, risk level, anomaly score, and raw log content, enabling timely security response.

Dashboard and Visualization

To provide real-time visibility into the detected security events, a Streamlit based dashboard is integrated with the bot. The dashboard reads event analysis results from a continuously updated CSV file generated by the bot. This file stores timestamps, user identifiers, assigned risk levels, cluster IDs, and Isolation Forest scores for each processed event. The dashboard automatically refreshes at short intervals, ensuring that newly detected events and alerts are displayed with minimal delay.

The dashboard presents key security metrics such as the total number of processed events, the number of high-risk alerts, the number of unique users, and the average anomaly score. It also displays recent alerts in a tabular view and visualizes user activity and anomaly distributions using charts and scatter plots. By combining numerical metrics with interactive visualizations, the dashboard allows security analysts to quickly understand system behavior, identify high-risk users, and monitor trends in anomalous activity in real time.

Insights

The analysis shows that combining clustering and anomaly detection provides an effective approach for identifying suspicious endpoint behavior. The Isolation Forest model successfully highlights rare and unusual events, while K-Means clustering adds contextual risk levels that help distinguish

between normal, moderate, and high-risk activities. This combination reduces false positives and allows better prioritization of security alerts, especially when analyzing large volumes of endpoint logs.

Additionally, the integration of user and IP-based analysis with graph network modeling offers deeper visibility into system behavior. Central users or IP addresses with high activity and strong connectivity were often associated with higher risk levels or anomaly scores. These insights demonstrate that unsupervised learning techniques, when applied consistently to raw endpoint logs, can effectively support real-time security monitoring and assist in early detection of potential threats.

Conclusion

This project demonstrates the effective use of machine learning and graph-based analysis to detect anomalies in raw endpoint logs. The implemented system successfully combines clustering, anomaly detection, and real-time monitoring to identify high-risk activities. Overall, the results show that the approach is practical, scalable, and well-suited for endpoint security analysis.